Document Loaders — CSVLoader

1. Today's Master Formula

CSV File

↓

CSVLoader

↓

One Document Per Data Row

This is the most important idea of today's topic.

2. Real-Life Situation

Suppose our training institute maintains student information in a CSV file.

Example:

ID Name Course Fee

S101 Ravi Python 5000

S102 Priya Java 6000

S103 Rahul Data Science 12000

S104 Shiva Agentic AI 18000

Now suppose Ravi asks:

Sir, what is my course fee?

We do not want to mix all students' information together.

Each student's data is separate.

That is why **row-wise representation** is useful.

3. What Is CSVLoader?

Definition

CSVLoader is a LangChain Document Loader that reads CSV files and creates one Document object for each CSV data row.

Simple flow:

CSV File

↓

CSVLoader

↓

Row 1 → Document 1

Row 2 → Document 2

Row 3 → Document 3

Important:

Number of Documents

=

Number of Data Rows

The header row is not counted as a data row.

4. Example CSV

Suppose:

student_id,name,course,fee

S101,Ravi,Python,5000

S102,Priya,Java,6000

S103,Rahul,Data Science,12000

Then:

Header

↓

Not a Document

S101 Row

↓

Document 1

S102 Row

↓

Document 2

S103 Row

↓

Document 3

5. How Does CSVLoader Prepare page_content?

Suppose one CSV row contains:

Student ID: S101

Name: Ravi

Course: Python

Fee: 5000

CSVLoader typically converts the row into labelled text.

Conceptually:

student_id: S101

name: Ravi

course: Python

fee: 5000

This complete labelled text becomes:

document.page_content

6. Why Are Column Names Useful?

CSV data may originally look like:

S101,Ravi,Python,5000

But labelled text is easier to understand:

student_id: S101

name: Ravi

course: Python

fee: 5000

So:

Raw Row

↓

Field Names + Values

↓

Meaningful Text Representation

Main conclusion:

Each CSV row becomes a Document whose page_content describes that row's field values.

7. Understanding CSV Metadata

For PDF documents we saw metadata such as:

source

page

total_pages

For CSV loading, an important metadata field is:

row

Example for first data row:


```
{
   "source": "students.csv",
   "row": 0
}
```


Second data row:


```
{
   "source": "students.csv",
   "row": 1
}
```


Third data row:


```
{
   "source": "students.csv",
```

```
  "row": 2
}
```

8. Very Important — Row Index Starts from Zero

Actual Data Row metadata["row"]

First Data Row 0

Second Data Row 1

Third Data Row 2

Important:

Header

↓

Not Counted as Row 0

So:

row = 0

means:

First CSV Data Row

9. Important Metadata Fields

source

↓

Which CSV file?

row

↓

Which data row?

Example:

```
{
   "source": "students.csv",
   "row": 0
}
```

10. Understanding metadata_columns

metadata_columns tells CSVLoader which CSV columns should be moved into Document.metadata.

Example CSV:

```
student_id,name,course,fee
S101,Ravi,Python,5000
```

Suppose we want:

page_content

---

name: Ravi

fee: 5000

and:

```
metadata = {
   "student_id": "S101",
   "course": "Python"
}
```

Then:

```
metadata_columns=[
   "student_id",
   "course"
]
```

11. Why Use metadata_columns?

It helps separate:

Main Content

↓

page_content

from:

Filtering / Identification Information

↓

metadata

Example:

Student Name + Fee

↓

page_content

Student ID + Course

↓

metadata

12. Understanding content_columns

content_columns tells CSVLoader which CSV fields should appear inside page_content.

Suppose CSV contains:

student_id

name

course

fee

But we want only:

name

fee

inside page_content.

Then:

```
content_columns=[
    "name",
    "fee"
]
```

13. metadata_columns vs content_columns

Parameter	Purpose
metadata_columns	Fields stored in metadata
content_columns	Fields included in page_content

Important:

A field can appear in both if selected for both purposes.

14. Easy Memory Formula

metadata_columns

↓

Document.metadata

content_columns

↓

Document.page_content

15. Understanding source_column

By default, source metadata identifies the CSV file.

Example:

```
{
   "source": "students.csv"
}
```

But we can tell CSVLoader to use a CSV column as the source.

Example:

source_column="student_id"

Then:

Ravi

↓

source = S101

Priya

↓

source = S102

16. Why Use source_column?

Suppose each row has its own unique identifier.

Example:

student_id

S101

S102

S103

Then using:

source_column="student_id"

makes the source metadata row-specific.

17. metadata_columns vs source_column

These are different.

metadata_columns

Adds selected fields

to metadata

under their own names

Example:

```
{
   "course": "Python"
```

}

source_column

Specifically controls

metadata["source"]

Example:

{

"source": "S101"

}

18. Understanding csv_args

Not every CSV file uses a comma.

Some files may use:

;

as a delimiter.

Example:

student_id;name;course;fee

S101;Ravi;Python;5000

Then we use:

csv_args={

"delimiter": ";"

}

csv_args lets us configure how the CSV reader interprets the file structure.

19. Important CSVLoader Syntax Summary

Requirement	**Syntax**
Import CSVLoader	from langchain_community.document_loaders import CSVLoader
Create loader	CSVLoader(file_path="students.csv")
Select metadata columns	metadata_columns=["course"]
Select content columns	content_columns=["name", "fee"]
Change source column	source_column="student_id"
Configure CSV parsing	csv_args={"delimiter": ";"}

20. Program 1 — Basic CSVLoader Program

```
from langchain_community.document_loaders import (
    CSVLoader
)



# ========================================
# CREATE CSV LOADER
# ========================================

loader = CSVLoader(
    file_path="students.csv",
    encoding="utf-8"
)



# ========================================
# LOAD CSV DOCUMENTS
# ========================================
```

```
documents = loader.load()



# ========================================
# DISPLAY DOCUMENT COUNT
# ========================================

print("TOTAL DOCUMENTS")
print("---------------")

print(
    len(documents)
)



# ========================================
# DISPLAY EACH DOCUMENT
# ========================================

for document in documents:
```

```
print("\nPAGE CONTENT")

print(
    document.page_content
)

print("METADATA")

print(
    document.metadata
)
```

21. Program 1 Flow

```
students.csv
    ↓
CSVLoader
    ↓
loader.load()
    ↓
```

List of Documents

↓

Loop

↓

page_content

+

metadata

22. What Does loader.load() Return?

documents = loader.load()

returns:

List of Document Objects

Example:

students.csv

3 Data Rows

↓

documents

↓

[

Document 1,

```
    Document 2,
    Document 3
]
```

23. Program 1A — Create and Load an Actual CSV File

Objective

Read an actual CSV file and observe how each data row becomes a LangChain Document.

```
from pathlib import Path

from langchain_community.document_loaders import (
    CSVLoader
)



# ========================================
# CREATE SAMPLE CSV FILE
# ========================================

file_path = (
```

```
    Path(__file__).resolve().parent
    / "students_day4.csv"
)


file_path.write_text(
    "student_id,name,course,fee\n"
    "S101,Ravi,Python,5000\n"
    "S102,Priya,Java,6000\n"
    "S103,Rahul,Data Science,12000\n",
    encoding="utf-8"
)


# ========================================
# CREATE CSV LOADER
# ========================================

loader = CSVLoader(
    file_path=str(file_path),
```

```
    encoding="utf-8"
)



# ========================================
# LOAD CSV DOCUMENTS
# ========================================

documents = loader.load()



# ========================================
# DISPLAY DOCUMENT COUNT
# ========================================

print("TOTAL DOCUMENTS")
print("---------------")

print(
    len(documents)
```

```
)


# ========================================
# DISPLAY EACH DOCUMENT
# ========================================

for number, document in enumerate(
    documents,
    start=1
):

    print(
        "\nDOCUMENT",
        number
    )

    print(
        "---------------"
    )
```

```
print(
    "PAGE CONTENT"
)

print(
    document.page_content
)

print(
    "\nSOURCE:"
)

print(
    Path(
        document.metadata["source"]
    ).name
)

print(
```

```
    "ROW INDEX:",
    document.metadata["row"]
)
```

24. Expected Structure

For:

student_id,name,course,fee

S101,Ravi,Python,5000

S102,Priya,Java,6000

S103,Rahul,Data Science,12000

we get:

3 Data Rows

↓

3 Documents

Possible first Document:

page_content

---

student_id: S101

name: Ravi

course: Python

fee: 5000

Metadata:

```
{
   "source": "...students_day4.csv",
   "row": 0
}
```

25. Understanding enumerate()

```
for number, document in enumerate(
   documents,
   start=1
):
```

This gives:

1 → Document 1

2 → Document 2

3 → Document 3

start=1 makes displayed numbering begin from 1.

26. Display Number vs Row Index

Be careful.

Displayed Document Number

↓

Starts at 1

But:

metadata["row"]

↓

Starts at 0

Example:

DOCUMENT 1

row = 0

DOCUMENT 2

row = 1

DOCUMENT 3

row = 2

27. Program 2 — metadata_columns and content_columns

Objective

Separate selected CSV fields into:

page_content

and:

metadata

```
from pathlib import Path

from langchain_community.document_loaders import (
    CSVLoader
)



# ========================================
# CREATE CSV FILE
# ========================================

file_path = (
    Path(__file__).resolve().parent
    / "student_metadata_day4.csv"
```

```
)


file_path.write_text(
    "student_id,name,course,fee\n"
    "S101,Ravi,Python,5000\n"
    "S102,Priya,Java,6000\n",
    encoding="utf-8"
)


# ========================================
# CREATE CSV LOADER
# ========================================

loader = CSVLoader(
    file_path=str(file_path),
    encoding="utf-8",

    metadata_columns=[
```

```
        "student_id",
        "course"
    ],

    content_columns=[
        "name",
        "fee"
    ]
)



# ========================================
# LOAD DOCUMENTS
# ========================================

documents = loader.load()



# ========================================
# DISPLAY RESULTS
```

```
# ========================================

print(
    "TOTAL DOCUMENTS:",
    len(documents)
)


for number, document in enumerate(
    documents,
    start=1
):

    print(
        "\nDOCUMENT",
        number
    )

    print(
        "----------------"
```

```
)

print(
    "PAGE CONTENT"
)

print(
    document.page_content
)


print(
    "\nSTUDENT ID:"
)

print(
    document.metadata[
        "student_id"
    ]
)
```

```
print(
   "COURSE:"
)

print(
   document.metadata[
      "course"
   ]
)


print(
   "ROW INDEX:"
)

print(
   document.metadata[
      "row"
```

```
        ]
    )
```

28. Program 2 Result

For Ravi:

PAGE CONTENT

---

name: Ravi

fee: 5000

Metadata contains:

```
{
    "student_id": "S101",
    "course": "Python",
    "row": 0,
    ...
}
```

So:

student_id

course

    ↓

metadata

and:

name

fee

↓

page_content

29. Important Observation

Even when we use:

metadata_columns

and:

content_columns

CSVLoader still provides useful source and row information.

30. Program 3 — source_column and csv_args

Objective

1. Read a semicolon-separated file.
2. Use a CSV field as the source metadata.
3. Store another field in metadata.

from pathlib import Path

```
from langchain_community.document_loaders import (
    CSVLoader
)


# ========================================
# CREATE SEMICOLON-SEPARATED FILE
# ========================================

file_path = (
    Path(__file__).resolve().parent
    / "special_students_day4.csv"
)


file_path.write_text(
    "student_id;name;course;fee\n"
    "S101;Ravi;Python;5000\n"
    "S102;Priya;Java;6000\n",
```

```
    encoding="utf-8"
)


# ========================================
# CREATE CSV LOADER
# ========================================

loader = CSVLoader(
    file_path=str(file_path),
    encoding="utf-8",

    source_column="student_id",

    metadata_columns=[
        "course"
    ],

    csv_args={
        "delimiter": ";"
```

```
    }
)


# ========================================
# LOAD DOCUMENTS
# ========================================

documents = loader.load()


# ========================================
# DISPLAY RESULTS
# ========================================

print(
    "TOTAL DOCUMENTS:",
    len(documents)
)
```

```
for number, document in enumerate(
    documents,
    start=1
):

    print(
        "\nDOCUMENT",
        number
    )

    print(
        "----------------"
    )

    print(
        "PAGE CONTENT"
    )

    print(
```

```
    document.page_content
)



print(
    "\nSOURCE:"
)

print(
    document.metadata[
        "source"
    ]
)



print(
    "COURSE:"
)

print(
```

```
        document.metadata[
            "course"
        ]
    )
```

31. Observation 1 — csv_args

Our file uses:

;

instead of:

,

So we use:

```
csv_args={
    "delimiter": ";"
}
```

Therefore CSVLoader correctly understands the fields.

32. Observation 2 — source_column

We use:

source_column="student_id"

Therefore:

Ravi

↓

source = S101

and:

Priya

↓

source = S102

33. Observation 3 — metadata_columns

We selected:

metadata_columns=[

"course"

]

Therefore:

course

↓

metadata

Example:

```
{
   "course": "Python"
}
```

34. What Happens When content_columns Is Not Supplied?

In Program 3, content_columns was not supplied.

According to the source notes:

CSVLoader automatically includes the other non-metadata fields in page_content.

So conceptually:

metadata fields

↓

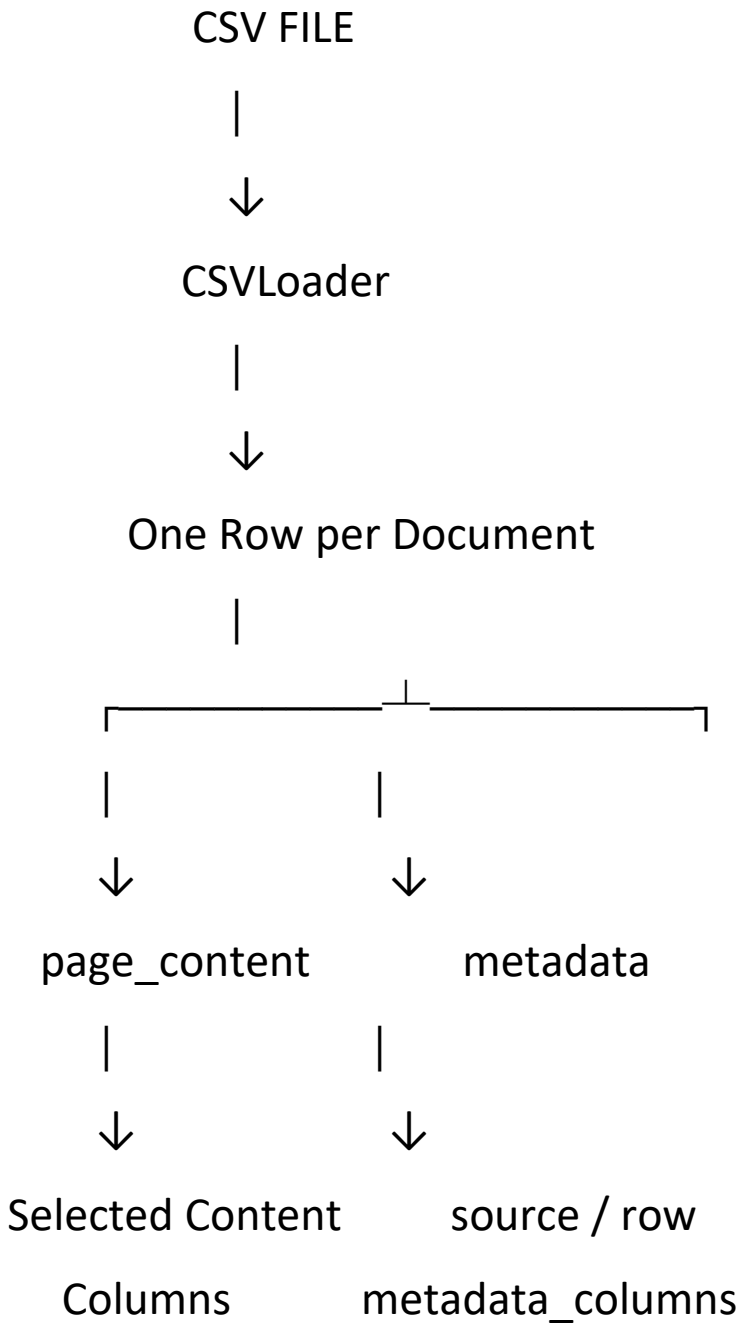
Removed from main content selection

Remaining fields

↓

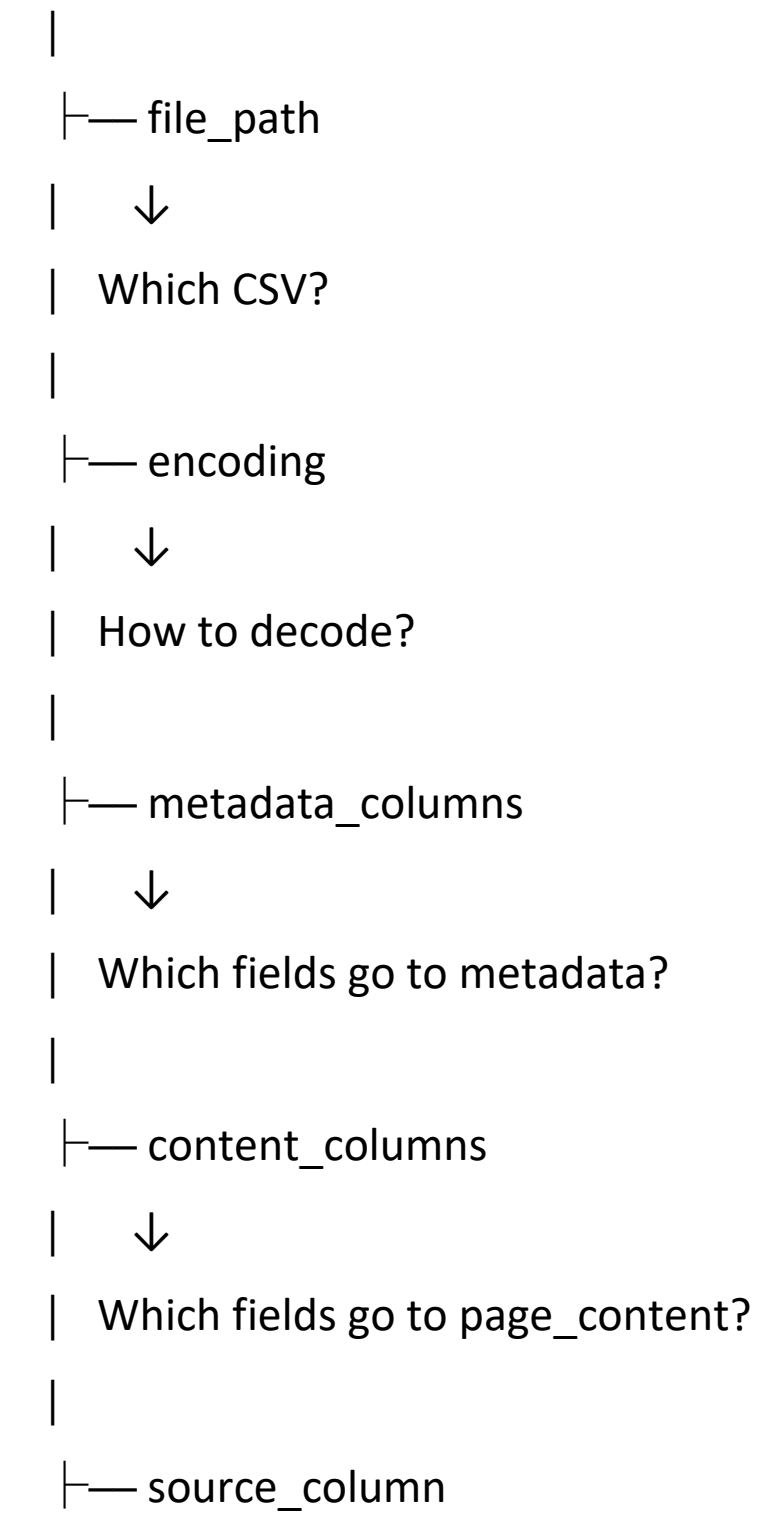
page_content

35. Complete CSVLoader Architecture


CSV FILE
CSVLoader
One Row per Document
page_content
metadata
Selected Content Columns
source / row metadata_columns

36. Parameter Architecture

CSVLoader



```
|    ↓
|  Which field becomes source?
|
└── csv_args
      ↓
    How is CSV structured?
```

37. TextLoader vs PyPDFLoader vs CSVLoader

Loader	Source	Typical Document Representation
TextLoader	.txt	Usually one Document per text file
PyPDFLoader	.pdf	Can be one Document per page
CSVLoader	.csv	One Document per data row

Easy memory:

TXT

↓

TextLoader

PDF

↓

PyPDFLoader

CSV

↓

CSVLoader

38. Most Important Difference

For CSV:

One File

↓

Many Rows

↓

Many Documents

Example:

students.csv

↓

4 Student Rows

↓

4 Documents

39. Common Mistake — Counting Header as a Document

Wrong:

Header + 3 Data Rows

↓

4 Documents

Correct:

3 Data Rows

↓

3 Documents

The header supplies column names.

40. Common Mistake — Thinking Row Index Starts at 1

Wrong:

First Row

↓

row = 1

Correct:

First Data Row

↓

row = 0

41. Common Mistake — Thinking metadata_columns Controls page_content

No.

metadata_columns

↓

metadata

Whereas:

content_columns

↓

page_content

42. Common Mistake — Thinking source_column Adds a Normal Metadata Field

source_column specifically changes:

metadata["source"]

It does not behave exactly like metadata_columns.

43. Common Mistake — Assuming Every CSV Uses Comma

Some files may use:

;

or another supported delimiter.

Then we configure:

```
csv_args={
   "delimiter": ";"
}
```

44. Five Important Coding Lines

Import

```
from langchain_community.document_loaders import (
   CSVLoader
)
```

Basic Loader

```
loader = CSVLoader(
   file_path="students.csv"
)
```

Load Documents

```
documents = loader.load()
```

Read Content

```
document.page_content
```

Read Row Metadata

```
document.metadata["row"]
```

45. Advanced Configuration Revision

```
loader = CSVLoader(
    file_path="students.csv",

    metadata_columns=[
        "student_id",
        "course"
    ],

    content_columns=[
        "name",
        "fee"
    ],

    source_column="student_id",

    csv_args={
        "delimiter": ","
    }
)
```

Memory:

metadata_columns

↓

Metadata

content_columns

↓

Page Content

source_column

↓

metadata["source"]

csv_args

↓

CSV Parsing Rules

46. Final Revision Points

1. CSVLoader reads CSV files.
2. Each CSV data row becomes one LangChain Document.
3. The header is not treated as a data row.
4. page_content contains labelled field values.
5. metadata["row"] is a zero-based data-row index.

6. metadata["source"] normally identifies the CSV file.
7. metadata_columns moves selected fields into metadata.
8. content_columns selects fields for page_content.
9. source_column controls the source metadata value.
10. csv_args controls CSV parsing options such as delimiter.

47. Final Master Formula

CSV FILE

↓

CSVLoader

↓

DATA ROW

↓

Document

│

├── page_content

│

└── metadata

And:

metadata_columns

↓

Metadata

content_columns

↓

Page Content

source_column

↓

Source

csv_args

↓

CSV Format Rules

48. One-Line Conclusion

CSVLoader converts each CSV data row into a LangChain Document, where selected fields can be placed in page_content, selected fields can be stored in metadata, and row/source information helps identify where the data came from.